

MT5 CLIENT LIVE MONITOR

Backend Development Specification Phase 1 — MT5 Connectivity, Client Search & Open Positions

Implementation handoff document for the coding AI

Project	MT5 Client Live Monitor
Current repository	https://github.com/SM71234/MT5_Monitor.git
MT5 reference repository	https://github.com/SM71234/AI-MT5-Logs-Analyser.git
Scope	Backend foundation: real MT5 read-only integration
Status	Phase 1-5 implementation specification

1. Executive Intent

The frontend of MT5 Client Live Monitor is already implemented. This document defines the backend work required to connect that UI to real MetaTrader 5 broker infrastructure through the MT5 Manager API. The first milestone is deliberately narrow: establish a secure broker connection, reuse the connection, search real trading accounts, retrieve account information, and retrieve current open positions.

Primary milestone

Broker → MT5 Manager connection → real client/account search → account details → real open positions → existing UI.

Telegram, web push, email, trade-event detection and monitoring automation are deferred until the read-only data path has been proven against a real broker.

2. Existing Product Context

- Authentication and sign-out.
- Mobile-first bottom navigation and desktop sidebar.
- Light/dark theme with persistence.
- Broker summary and monitored-client dashboard cards.
- Saved brokers and active broker selection.
- Client search by login, name or group.
- Recently viewed clients.
- Client profile with Balance, Equity, Margin and Floating P/L.
- Expandable live positions list.
- Monitor/unmonitor bell toggle.
- Monitored clients/watchlist grouped by broker.
- Notification center with event filters and detailed alert view.
- Settings for Push, Telegram and Email delivery channels.
- Read-only monitoring notice and status indicators.

3. Mandatory Reference Material

Before coding, study both repositories and the integration guide. The reference project is an implementation reference, not something to copy blindly.

- Current: https://github.com/SM71234/MT5_Monitor.git
 - Reference: <https://github.com/SM71234/AI-MT5-Logs-Analyser.git>
 - MT5_INTEGRATION_GUIDE.md in the reference project.
1. Determine how the reference establishes an MT5 Manager API connection.
 2. Determine how broker/server, Manager login and password are handled.
 3. Determine how Manager sessions are created and reused.
 4. Determine how client/account data is obtained.
 5. Determine how MT5 data is normalized.
 6. Determine how errors and connection lifecycle are handled.
 7. Separate reusable MT5 connectivity from AI Journal Analyzer-specific logic.

No guessing

Do not invent MT5 Manager API methods or fields. Verify each method and field against the integration guide, reference implementation and actual SDK.

4. Permanent Read-Only Safety Rule

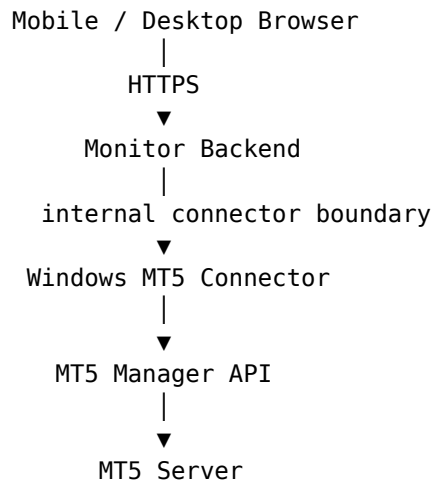
This application is strictly a monitoring and notification system.

- Never open, close or modify positions/orders.
- Never modify SL, TP, volume or price.
- Never create, cancel or modify orders.
- Never execute dealer/trading operations.
- Never expose a trading/execution endpoint.
- Never create code that can be repurposed into a trade command.

Allowed direction

MT5 → READ → MONITOR → DISPLAY → NOTIFY. The reverse direction must never contain trading commands.

5. Target Backend Architecture



The browser must never load the native Manager DLL, directly connect to the MT5 server, or receive Manager credentials. Windows-specific integration stays isolated in the connector.

6. Development Chunks

Chunk	Scope	Completion condition
1	MT5 connector & broker connection	Real Manager API connection works.
2	Broker session management	Connection is reused and safely

		recovered.
3	Client/account search	Real login/name/group search works.
4	Account details	Real account metrics populate DTOs.
5	Open positions	Real positions populate DTOs.
6	Frontend integration	Existing UI uses real backend data.
7+	Monitoring/events/notifications	Deferred until explicitly authorized.

7. Chunk 1 — MT5 Broker Connectivity

```

User
↓
Add Broker
↓
Backend validation
↓
Windows MT5 Connector
↓
MT5 Manager API
↓
MT5 Server
↓
Connection status

```

7.1 Broker configuration

- Broker ID
- Broker display name
- MT5 server address/name
- Manager login
- Encrypted Manager password
- Connection status
- Created/updated timestamps

Use the exact fields required by the verified connection implementation.

7.2 Connection states

- DISCONNECTED
- CONNECTING
- CONNECTED
- RECONNECTING
- ERROR

7.3 Expected endpoints

```

POST    /api/brokers
GET     /api/brokers

```

```

GET    /api/brokers/:id
PUT    /api/brokers/:id
DELETE /api/brokers/:id
POST   /api/brokers/:id/test-connection
GET    /api/brokers/:id/status

```

8. Credential Security

- Never store broker passwords in plaintext.
- Never return decrypted passwords to the browser.
- Never put credentials in localStorage.
- Never log passwords, tokens or encryption keys.
- Never commit credentials to Git.
- Keep encryption keys separate from encrypted credentials.
- Design the key layer so production KMS/HSM/secrets management can be introduced later.

9. Chunk 2 — Broker Session Manager

Do not establish a new Manager API connection for every HTTP request.

```

Broker A → persistent Manager Session A
Broker B → persistent Manager Session B

```

- getSession(brokerId)
- connect(brokerId)
- disconnect(brokerId)
- reconnect(brokerId)
- getStatus(brokerId)
- Prevent duplicate concurrent connection attempts.
- Reuse sessions across read requests.
- Handle failure and controlled reconnection.

10. MT5 Broker-Server Load Protection

The system cannot honestly promise zero broker-server load. Every Manager API connection/query consumes resources. The requirement is to minimize unnecessary load.

- Reuse persistent Manager API sessions.
- Avoid reconnecting on every request.
- Avoid querying the entire broker for every client lookup.
- Cache suitable non-volatile account information for short periods.
- Deduplicate identical in-flight requests where appropriate.
- Bound concurrent Manager API operations.
- Use retry limits and exponential backoff.
- Do not create one MT5 session per browser user for the same broker unless required by the API.
- Do not poll every client every second.
- Prepare for MT5 trade-pump/event mechanisms where appropriate.

Design target

Many browser requests must not translate into the same number of MT5 connections or uncontrolled MT5 API calls.

11. Chunk 3 — Client / Trading Account Search

```

Selected Broker
  ↓
Broker MT5 Session
  ↓
MT5 user/account lookup
  ↓
Normalized ClientAccount DTO
  ↓
Existing Client Search UI

```

- Search by MT5 login
- Search by client name
- Search by group

Verify broker authorization before querying MT5.

12. Chunk 4 — Client Account Details

The Client Profile UI requires real account information from MT5.

Field	Purpose	Verification rule
Login	MT5 account identifier	Verify from Manager API user/account structure.
Name	Client display name	Use actual MT5 field.
Group	Trading group	Use actual MT5 field.
Balance	Account balance	Prefer authoritative MT5 value.
Equity	Current equity	Prefer authoritative MT5 value.
Margin	Used margin	Prefer authoritative MT5 value.
Floating P/L	Unrealized P/L	Use authoritative MT5 value where available.
Leverage	Account leverage	Only return if verified available.

12.1 Normalized DTO

```

interface ClientAccount {
  login: number;
  name: string;
  brokerId: string;
  group?: string;
  balance?: number;
}

```

```

equity?: number;
margin?: number;
floatingProfit?: number;
leverage?: number;
}

```

Adjust to the verified Manager API structure. Do not expose raw SDK objects.

13. Chunk 5 — Open Position Retrieval

The integration guide/reference material identifies active-position retrieval through Manager API position functionality, including the documented pattern `PositionGet(login_id)`. Verify the exact SDK method and returned structure before implementation.

13.1 Position fields

Field	Required	Notes
Position ID	Yes	Important for future event correlation.
Symbol	Yes	Instrument symbol.
Direction	Yes	Normalize to BUY / SELL.
Volume / Lots	Yes	Actual position volume.
Open Price	Yes	Actual opening price.
Current Price	Yes	Current market-side price if available.
Profit	Yes	Current floating position P/L.
Stop Loss	Conditional	Only if verified from actual Manager API structure.
Take Profit	Conditional	Only if verified from actual Manager API structure.
Event/time data	Future	Prepare for later monitoring/event correlation.

13.2 Position endpoint

GET `/api/brokers/:brokerId/clients/:login/positions`

```

{
  "clientLogin": 905672,
  "positions": [
    {
      "positionId": "84592341",
      "symbol": "EURUSD",
      "direction": "BUY",
      "volume": 2.0,
      "openPrice": 1.16520,
      "currentPrice": 1.16610,
      "profit": 180.00,

```

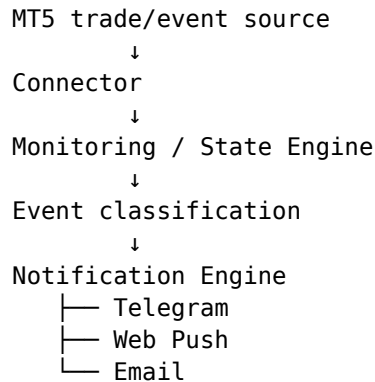
```

    "sl": 1.16000,
    "tp": 1.17500
  }
]
}

```

The JSON above is an illustrative contract. Actual field availability and naming must follow the verified SDK.

14. Future Monitoring Architecture — Prepare, Do Not Implement



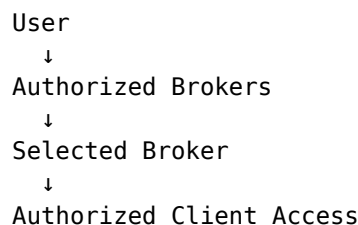
The integration guide references PUMP_MODE_TRADES / PUMP_MODE_FULL for trade synchronization. Investigate the exact supported mechanism and threading requirements before choosing event-driven processing or polling-plus-diff.

15. Thread Safety & Concurrency

- Inspect the actual SDK/reference implementation for thread-safety constraints.
- Do not concurrently invoke unsafe operations against one Manager API session.
- Use a lock, queue or serialized access strategy where required.
- Bound concurrency across brokers and sessions.
- Do not let frontend request volume directly determine MT5 operation concurrency.

16. Authentication & Authorization

- Use the authentication system already present in the Monitor project.
- Do not create a second unrelated login system.
- Every broker/client request must verify user access.
- Do not trust brokerId or clientLogin from the browser without authorization checks.
- Keep the initial authorization model simple but extensible to per-user broker/client access.



↓
MT5 read operation

17. Database & Persistence

Use the database architecture already present in the Monitor project. Do not introduce a second unrelated database without a concrete technical reason.

- Persist broker configuration and secure credential references.
- Persist user-to-broker access relationships as required.
- Do not persist full live position streams in this milestone unless necessary.
- Treat MT5 as the source of truth for current account/position state.

18. Initial API Contract

Method	Endpoint	Purpose	MT5 effect
POST	/api/brokers	Save broker configuration	None
GET	/api/brokers	List authorized brokers	Read/application state
POST	/api/brokers/:id/test-connection	Validate MT5 connection	Read-only
GET	/api/brokers/:id/status	Connection status	Read-only
GET	/api/brokers/:id/clients/search	Search MT5 accounts	Read-only
GET	/api/brokers/:id/clients/:login	Get account details	Read-only
GET	/api/brokers/:id/clients/:login/positions	Get current positions	Read-only

19. Error Handling

- Invalid broker credentials → controlled connection error.
- Broker/MT5 server unavailable → controlled unavailable state.
- Manager session expired → controlled reconnect.
- Client not found → empty result, not generic server failure.
- Unauthorized broker → authorization error.
- Position retrieval failure → controlled data-unavailable state.
- Connector unavailable → backend dependency error.
- Timeout → bounded retry/backoff.

```
{
  "error": "BROKER_UNAVAILABLE",
  "message": "The broker connection is currently unavailable."
}
```

20. Logging & Observability

- Broker connection attempt/success/failure.
- Reconnect attempt and final status.
- Client search and found/not-found result.
- Position retrieval success/failure.
- Connector and Manager API failures.
- Use safe identifiers such as brokerId, clientId and requestId.
- Never log passwords, Telegram tokens, access tokens, encryption keys or secrets.

21. Testing & Acceptance

21.1 Broker tests

- Add broker
- Secure credential storage
- Successful connection
- Invalid credentials
- Broker unavailable
- Session reuse
- Reconnect after failure

21.2 Client tests

- Search by login
- Search by name
- Search by group
- Client found
- Client not found
- Unauthorized broker blocked
- Account details retrieved

21.3 Position tests

- Zero positions
- One position
- Multiple positions
- BUY
- SELL
- Volume
- Symbol
- Open/current price
- Profit
- Position ID
- SL if supported
- TP if supported

22. Performance Requirements

- Do not create a new MT5 connection per HTTP request.

- Do not create one Manager session per website user for the same broker unless required by verified API constraints.
- Avoid redundant identical requests.
- Use bounded concurrency.
- Use appropriate short-lived caching for non-volatile data.
- Use backoff during repeated MT5 errors.
- Do not continuously poll every monitored client during this phase.

23. Required Developer Deliverable

1. Final backend architecture.
2. All files created.
3. All files modified.
4. Exact MT5 Manager API methods used.
5. How the Manager API DLL is loaded and isolated.
6. How sessions are created and reused.
7. How credentials are secured.
8. How client search works.
9. Exact account fields retrieved.
10. Exact position fields retrieved.
11. Whether SL/TP are actually available.
12. How MT5 load is controlled.
13. How errors/reconnection are handled.
14. How the existing frontend receives real data.
15. Actual tests performed against a real broker.
16. Known limitations.

24. Explicit Stop Condition

STOP AFTER DATA FOUNDATION

Once a real broker can be connected, a real client can be searched, real account details retrieved, and real open positions displayed, stop. Do not implement Telegram, trade-event detection or notification delivery without explicit authorization.

25. Final Safety Audit

- ☐ No trade execution endpoint
- ☐ No order execution endpoint
- ☐ No close-position function
- ☐ No modify-position function
- ☐ No SL/TP modification
- ☐ No volume modification

- ☐ No dealer operation
- ☐ No trading command
- ☐ No credentials in frontend
- ☐ No passwords in logs
- ☐ No secrets committed to Git
- ☐ Broker authorization enforced
- ☐ Manager sessions reused
- ☐ MT5 server load controlled

26. Definition of Done

```

Login
↓
Brokers
↓
Add real MT5 broker
↓
Test Connection
↓
CONNECTED
↓
Select Broker
↓
Search real MT5 client
↓
Client Profile
↓
Real Balance / Equity / Margin / Floating P/L
↓
Real Open Positions
↓
Position ID / Symbol / Direction / Volume /
Open Price / Current Price / Profit /
SL & TP where verified

```

The normal application path must use real MT5 data. If MT5 is unavailable, show a controlled unavailable/stale state rather than silently substituting fake trading data.

27. Subsequent Phases

Phase	Future capability	Dependency
6	Connect real data to production UI	Chunks 1-5
7	Monitored-client state engine	Stable position retrieval
8	Open/close/modify event	Verified event/pump or polling

	detection	design
9	Telegram notifications	Reliable event engine
10	Web Push / Email	Notification engine

READ-ONLY BY DESIGN

Observe • Monitor • Detect • Notify — Never Trade